BRACT's

Vishwakarma Institute of Information Technology, Pune Practical Implementation Sheet

Department: CS-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No:2 Roll no: 21 PRN No: 12411682

Subject Name: Deep Learning

Name of Student: Pranay Zunzunwala

Problem Statement: Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm:

Step 1: Import the required libraries such as NumPy, Pandas, Matplotlib, and the necessary modules from scikit-learn.

Step 2: Load the Wine dataset from the scikit-learn library and separate it into input features (**X**) and target labels (**y**).

Step 3: Split the dataset into training and testing sets using an 80:20 ratio.

Step 4: Standardize the training and testing feature data using StandardScaler to improve the performance of the neural network.

Step 5: Create a Multilayer Perceptron (MLP) classifier by specifying parameters such as the hidden layer size, activation function, optimizer, learning rate, and maximum iterations.

Step 6: Train the MLP classifier using the standardized training dataset.

Step 7: Use the trained MLP model to predict the class labels for the testing dataset.

Step 8: Evaluate the model by calculating the accuracy score to determine its overall classification performance.

Step 9: Generate and display the confusion matrix to compare the actual class labels with the predicted class labels.

Step 10: Display the classification report and visualize the comparison between the actual (y_test) and predicted (y_pred) labels, then analyze the results and conclude the model's performance.

Code :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import(accuracy_score, confusion_matrix, classification_report,
ConfusionMatrixDisplay)
wine = load_wine()
X,y = wine.data, wine.target
print("Feature Name:")
print(wine.feature_names)
print("\n")
print("Target Name:")
print(wine.target_names)
print("\n")
df = pd.DataFrame(X, columns = wine.feature_names)
df["Target"]=y

X_train, X_test, y_train, y_test= train_test_split(X, y, test_size=0.2, random_state=5, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

mlp = MLPClassifier(hidden_layer_sizes=(100,10), activation="relu", solver="adam")
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy of the Model is: ", accuracy,"%")

cm=confusion_matrix(y_test, y_pred)
print("Confusion Matrix for the above model: ")
print(cm)

disp = ConfusionMatrixDisplay(confusion_matrix = cm, display_labels=wine.target_names)
```

```
disp.plot(cmap="Blues")
plt.title("Confusion Matrix: ")
plt.show()

print(classification_report(y_test, y_pred, target_names = wine.target_names))

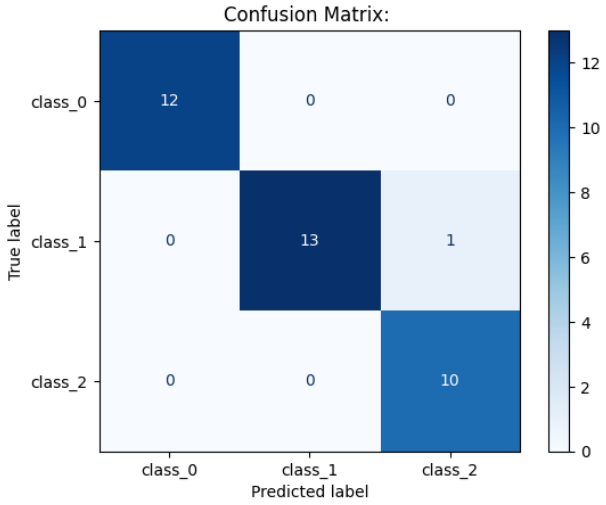
plt.figure(figsize=(8,6))
plt.scatter(range(len(y_test)), y_test, color="blue", label="Actual", marker="o")
plt.scatter(range(len(y_pred)), y_pred, color="red", label="Pedicted", marker="x")

plt.xlabel("Sample Index")
plt.ylabel("Class")
plt.title("Prediction vs Actual")
plt.yticks([0,1,2])
plt.legend()
plt.grid(True)
plt.show()
```

Output :

```
Accuracy of the Model is:  0.9722222222222222 %
```

```
Confusion Matrix for the above model:
[[12  0  0]
 [ 0 13  1]
 [ 0  0 10]]
```



	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	12
class_1	1.00	0.93	0.96	14
class_2	0.91	1.00	0.95	10
accuracy			0.97	36
macro avg	0.97	0.98	0.97	36
weighted avg	0.97	0.97	0.97	36

